

Operating System: Unix

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction to Unix/Linux
- **Session 2:** Installation of Linux (WSL, VirtualBox, Mac)
- **Session 3:** Linux Architecture & File Systems
- **Session 4:** CLI Basics: Commands & Navigation
- **Session 5:** Advanced CLI: I/O & Text Processing
- **Session 6:** Shell Scripting I: Basics & Automation
- **Session 7:** Shell Scripting II: Advanced Scripts
- **Session 8:** Processes & Init Systems
- **Session 9:** Users, Groups & Permissions
- **Session 10:** Text Editors: vi/vim
- **Session 11:** Github ? Networking?
- **Session 12:** Remote Access, Downloads & Shell Customization

Remote Access, Downloads & Shell Customization

Session 12

Remote Access, Downloads & Shell Customization

Session 12

Objectives:

- **Use SSH to connect to remote systems**
- **Download and extract content using wget, curl, tar, and unzip**
- **Customize their shell with alias, environment variables, and .bashrc modifications**

SSH: Secure Shell

What is SSH?

A cryptographic network protocol that allows secure communication between two machines, even over untrusted networks.

It is commonly used for:

- Secure remote login to servers
- Safe file transfers
- Running commands on remote systems

SSH: Secure Shell

ssh [username]@[hostname or IP address]

- [username]: your remote server username
- [hostname or IP address]: the server's hostname or IP address.

Ex: ssh thomas@192.168.56.1

SSH: Secure Shell

On Windows

- check ip from ipconfig on windows or check ip from the wifi (properties)
- Optional features -> add opensshd-server
- Try Get-Service sshd , should see running otherwise: Start-Service sshd
- *net user thomas thomaspwd /add*
- *net user thomas /active:yes*

On your VM (Linux)

- *apt install openssh-server,*
- *systemctl enable ssh*
- *systemctl start ssh*

SSH: Secure Shell

On MAC:

- Check `/etc/ssh/sshd_config` and you should see `PasswordAuthentication yes`
If not change the line and then run
`sudo launchctl stop com.openssh.sshd`
`sudo launchctl start com.openssh.sshd`
- In general, network allow sharing

SSH: Secure Shell

Password input?

SSH does not allow `--password` on the command line (for security)

Use **sshpass** only for demo/testing:

```
sshpass -p 'pwd' ssh thomas@192.168.56.1
```

Secure Copy: scp

Secure file transfer is an essential skill for Linux system administration. The **scp** command (Secure Copy Protocol) allows you to **safely copy files** over a network by using **SSH encryption**.

Unlike basic tools that send data in plain text, scp encrypts everything—your password, files, and commands—protecting them from eavesdropping or interception.

```
scp file.txt user@host:/path/  
scp -r folder/ user@host:/home/user/
```

Demo: Send a file from Linux VM to Windows scp file.txt thomas@192.168.56.1:~/Desktop/

rsync

rsync is a powerful command-line tool used to **synchronize files and directories** between two locations — locally or over a network via SSH..

It only transfers differences between source and destination — making it much faster than scp for repeated transfers or large directories.

Feature	scp	rsync
Secure transfer	 SSH	 SSH
Sync only changed files	 No	 Yes
Resume support	 No	 Yes
Directory structure		
Compression	 No	 (-z)

rsync

Syntax:

rsync [options] source destination

Option

Meaning

-a

Archive mode: preserve permissions, timestamps, symbolic links, etc.

-v

Verbose: show progress

-z

Compress data during transfer (faster over network)

-r

Recursive: copy subdirectories

--delete

Delete files from destination that are no longer in the source

-Sync folder to remote machine:

rsync -avz my_project/ thomas@192.168.1.39:~/projects/

-Pull files from a remote machine:

rsync -avz thomas@192.168.1.39:~/projects/ .

rsync

demo:

- create two folders test1 and test2*
- create 3 files into test1*
- rsync all the files from test1 to test2*
- modify one file and redo rsync*

mkdir test1 test2

cd test1;touch file1.txt file2.txt file3.txt

rsync -av test1/ test2/

gedit file2.txt

rsync -av test1/ test2/

Downloading Files: wget and curl

wget – For simple file downloads

wget gnu.org/software/wget/manual/wget.txt

wget -O lecture.pdf <https://example.com/file123.pdf> #DL and rename

curl – For more control (headers, APIs). Working with REST APIs or inspecting HTTP headers

ex: curl -O <https://example.com/file.zip> # save file

curl <https://example.com/data.csv> -o data.csv

Downloading Files: wget and curl

```
curl -X POST -H "Content-Type: application/json" -d '{"name":"Thomas"}'  
https://api.example.com/create
```

Part

curl

-X POST

-H "Content-Type: application/json"

-d '{"name":"Thomas"}'

https://api.example.com/create

Meaning

The tool that sends the request

“I want to send data” (like submitting a form)

"I'm sending data in JSON format" (the modern language of the web)

The actual message: "My name is Thomas"

The address of the service you're talking to

Extracting Archives

Extract .tar and .zip archives

`tar -czvf archive.tar.gz folder/` # Create

`tar -xvf archive.tar.gz` # Extract

`unzip archive.zip` # Extract .zip file

Option

-c

-x

-v

-f

-z

-j

-t

Meaning

Create a new archive

Extract files from archive

Verbose: show file names during processing

File: specify archive file name

Use **gzip compression** (for .tar.gz)

Use **bzip2 compression** (for .tar.bz2)

List files inside an archive (without extracting)

Extracting Archives

Demo:

```
mkdir testdir && echo "Hello" > testdir/hello.txt  
tar -czvf testdir.tar.gz testdir/  
tar -tvf testdir.tar.gz  
tar -xvf testdir.tar.gz
```

Alias

Alias command useful to create custom shortcuts for frequently used commands.

- Temporary:

```
alias ll='ls -lrt'
```

- Permanent:

Open ~/.bashrc and add this line: `alias ll='ls -lrt'`

Then close and reopen terminal OR `source ~/.bashrc`

Others examples:

```
alias projects='cd ~/project'
```

PATH variable

Why can you run “ls” anywhere?

PATH variable

Why can you run “ls” anywhere?

Your own script (e.g. myscript.sh) only runs if you're in the same directory your \$PATH.

But system commands like ls, cd, work from anywhere.

PATH variable

Why can you run “ls” anywhere?

Your own script (e.g. myscript.sh) only runs if you're in the same directory your \$PATH.

But system commands like ls, cd, work from anywhere.

Because the shell automatically searches in all directories listed in \$PATH when you run a command without a path.

PATH variable

echo \$PATH

-> usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games

When you run a command:

- ➔ If it finds the command in one of those directories → it runs it
- ➔ If not → it shows “command not found”

PATH variable

demo:

Create a simple shell script
gedit hello.sh

```
#!/bin/bash  
echo "Hello!!."
```

Make the script executable
chmod +x hello.sh

Add the line to your .bashrc:

```
export PATH=$PATH:$HOME/XXXXXX  
source ~/.bashrc
```

Run it from ANYWHERE